



39. File System API

What is a File?

File Names

Inode Number

What does "i" stand for?

File API (Attempt 1) - inode 기반 파일 접근

Disadvantages

Paths

Example: read/etc/bashrc

1. 디렉토리의 inode를 먼저 읽는다

2. inode가 가리키는 데이터 블록을 읽는다

Directory Calls

Directory Creation

Directory Reading

Why Is There No writedir() System Call?

File API (Attempt 2)

Disadvantages

File Descriptor (fd)

Key Idea

Descriptor Object

Per-Process File Descriptor Table (Process FD Table)

Well-Known File Descriptors

Kernel Data Structures (xv6 kernel)

xv6 struct file Definition

Global Open File Table (전역 파일 테이블)

Sharing struct file Across Multiple Processes

When using dup(), dup2(), or fcntl(F_DUPFD)

When is the offset NOT shared?

Code Snippet

File API (Attempt 3)

Deleting Files

참조 제거 방법

Many File Systems

Communicating Requirements: fsync - 쓰기 버퍼링

rename

Example

Atomic File Update

정리하면

질문: "보고서.txt를 최종보고서.txt로 이름 바꾸기"

질문: "보고서.txt의 내용을 안전하게 업데이트"

Summary

What is a File?

- **Array of persistent bytes** that can be read/written
읽고 쓸 수 있는 1차원 바이트 배열
- **File system** consists of many files
 - Refers to **collection of files**
 - **Also refers to part of OS** that manages those files
- Files need **names to access correct one**

File Names

- Three types of names
 1. **Unique id: inode numbers**
 2. **Path**
 3. **File descriptor**

Inode Number

- Each file has **exactly one inode number**
- Inodes are **unique (at a given time) within file system**
- USB 연결해도 같은 inode일 수 있다?-
다른 file system
- Different file system may use the same number, numbers may be recycled after deletes

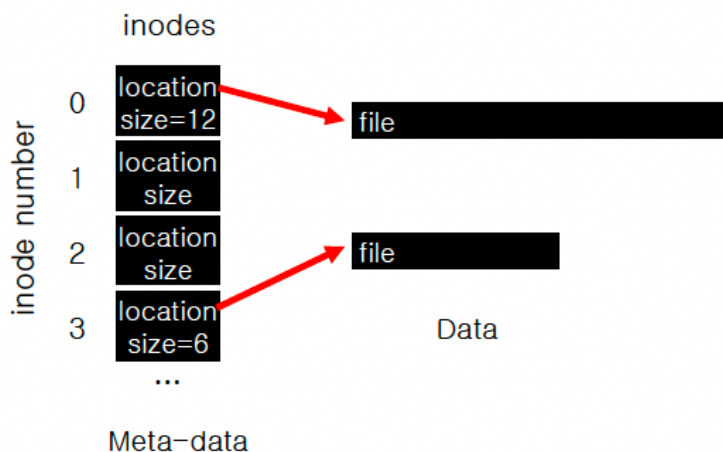
```
[youkim@fast0 ~]$ echo "foo.txt" > foo.txt
[youkim@fast0 ~]$ echo "bar.txt" > bar.txt
[youkim@fast0 ~]$ ls
bar.txt  foo.txt
[youkim@fast0 ~]$ ls -i
805777886 bar.txt  805777882 foo.txt
[youkim@fast0 ~]$ ls -il
total 8
805777886 -rw-rw-r-- 1 youkim youkim 8 May  2 14:18 bar.txt
805777882 -rw-rw-r-- 1 youkim youkim 8 May  2 14:17 foo.txt
[youkim@fast0 ~]$
```

- See inodes via "ls -li"; see them increment...

What does "i" stand for?

"In truth, I don't know either. It was just a term that we started to use. 'Index' is my best guess, because of the slightly unusual file system structure that stored the access information of files as a flat array on the disk..." ~ Dennis Ritchie

- The author of the books on the Unix operating system and the C programming language.
- Recipient of the 1983 Turing Award.



File API (Attempt 1) - inode 기반 파일 접근

- **read(int inode, void *buf, size_t nbyte)**
해당 inode 번호를 가진 파일에서 nbyte만큼 읽어서 buf에 저장하는 함수
- **write(int inode, void *buf, size_t nbyte)**
해당 inode 번호를 가진 파일에 nbyte만큼 buf 내용을 쓰는 함수.
- **seek(int inode, off_t offset)**
 - Seek does not cause disk seek until read/write
실제 디스크 물리적 이동(disk seek)은 일어나지 않음.
 - 오직 **파일의 논리적 읽기 위치 포인터**만 offset만큼 움직임
 - 실제 I/O는 다음 read/write가 호출될 때 발생

Disadvantages

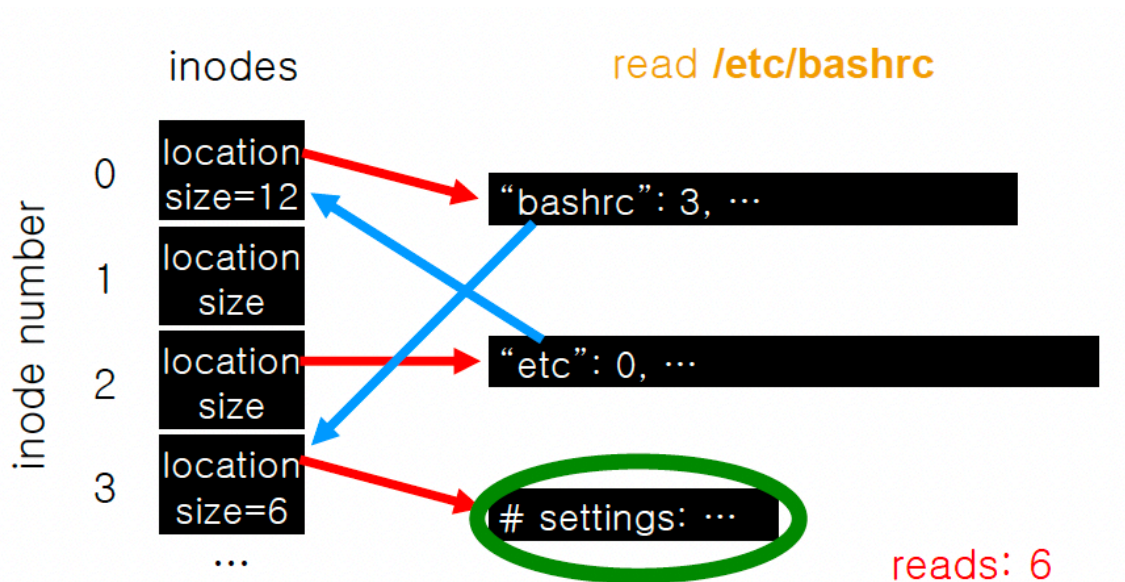
- Inode numbers are **difficult for users to remember**
사용자가 Inode number 기억하기 어렵다
- Inode numbers have **no inherent structure or meaning** and are not organized in a human-friendly way
inode 번호는 구조나 의미가 없다 (human-friendly 아님)
이름 구조나 경로 의미가 전혀 없음
- It is unclear how the offset semantics should work when multiple processes are accessing the same inode—each process may maintain its own offset
여러 프로세스가 같은 inode를 열 때 offset 의미가 모호해짐
inode로 파일을 직접 접근하면 **offset을 어디에 저장하는지가 문제**
👉 **UNIX는 이 문제를 해결하기 위해 "open → file descriptor 생성 → per-process offset 유지" 방식을 도입함**

Paths

- String-based names are much more user-friendly than numeric identifiers
문자열 기반 파일 이름(path)은 숫자 기반 inode보다 훨씬 인간 친화적
- The file system, however, still operates internally using inode numbers
파일 시스템 내부에서는 여전히 **inode 번호를 기반으로 동작**
- Maintain a mapping from path → inode inside a predefined "root" file (commonly inode 2)
파일 시스템은 path → inode 매핑을 유지해야 한다
 - It means that the information needed to translate a path string into an inode number is stored in the root directory (i.e., inode 2)
path를 inode로 변환하는 데이터 구조가 디렉터리(directory)
- This mapping structure is essentially a directory.
 - "/" → inode 2, "foo" → inode 5, "a.txt" → inode 17
- 정리?
 - 디렉터리: 이름 → inode 번호
 - inode: 파일의 실제 주소(블록 포인터), 크기, 권한, 시간 정보, link count ...
 - 데이터 블록: 실제 파일 내용
- Generalizing Path Management: Directory Trees

- Move beyond a single root mapping → use a **hierarchical directory tree**
- Only filenames need to be **unique within the same directory**
 - 예시)
 - /usr/dusseau/file.txt
 - /tmp/file.txt (Same filename, different directories → allowed)
- **Each directory stores its own file-to-inode mapping table (i.e., directory entries)**
 디렉토리마다 작은 매핑 테이블이 있는 게 O → 이를 "directory entry"(디렉토리 엔트리)라고 부름
- Key Idea
 - Instead of keeping all path → inode mappings in a single root file, each directory becomes a small lookup table mapping names to inode numbers
- **디렉토리 엔트리(dentry)** 안에 파일 이름이 저장됨
 - inode 자체는 이름을 저장하지 않음, inode는 *파일 메타데이터*와 블록 주소만 가지고 있음.

Example: read/etc/bashrc



디렉토리를 읽으려면:

1. 디렉토리의 inode를 먼저 읽는다

- 디렉토리도 결국 "파일"이기 때문에 inode가 있음.
- inode 안에는 다음 정보만 존재:
 - 파일 타입(디렉토리인지 파일인지)
 - 퍼미션(permission)
 - 파일 크기
 - 데이터 블록 위치 목록(= **directory entries** 가 저장된 블록 주소) ← 핵심

2. inode가 가리키는 데이터 블록을 읽는다

- 실제 파일 가서 파일안에 있는 디렉토리 엔트리 읽는 것
- `., ..` 등등

`read("/"):`

`read inode(2)`

`read data block containing {"etc":0, ...}`

`read("/etc"):`

`read inode(0)`

`read data block containing {"bashrc":3, ...}`

`read("/etc/bashrc"):`

`read inode(3)`

`read data block containing actual file contents`

Directory Calls

Directory Creation

- **`mkdir(path)`**

Creates a new directory and allocates a new inode for it

새로운 디렉토리를 위해 **새로운 inode를 할당**

Updates the parent directory to include a new name → inode entry

부모 디렉토리에 **"디렉토리 이름 → inode 번호"** 매핑 정보 추가

즉 부모 디렉토리의 디렉터리 엔트리가 수정되는 것까지 포함된 작업

Directory Reading

- **readdir(dir_inode)**

Reads and parses the directory's entries (file/directory names mapped to inode numbers)

특정 디렉터리 inode에 포함된 **디렉터리 엔트리들을 읽고 파싱**

즉, 해당 디렉터리에 어떤 파일/하위 디렉터리가 있는지 **파일명 → inode 번호의 리스트**를 읽는 것

Used by shells, file browsers, and system utilities to list contents

ls, 파일 탐색기 등등

Why Is There No writedir() System Call?

디렉터리 테이블 자체를 직접 쓰기 위한 전용 시스템 콜(writedir)이 없는 이유

- Because directories are implicitly written by other operations
디렉터리가 필요한 상황은 이미 다른 시스템 콜이 처리함
 - **Creating a file (creat, open(O_CREAT))** modifies its parent directory
파일 생성 (`creat` , `open(O_CREAT)`) → 부모 디렉터리에 엔트리 **추가**
 - **Deleting a file (unlink)** updates the directory table
파일 삭제 (`unlink`) → 디렉터리 엔트리 **삭제**
 - **Renaming / moving a file (rename)** alters directory entries
파일 이동/이름 변경 (`rename`) → 디렉터리 엔트리 **수정**
 - **Making a new directory (mkdir)** updates the parent directory
디렉터리 생성 (`mkdir`) → 부모 디렉터리에 엔트리 **추가**
- Directories are metadata structures; the OS updates them automatically when filesystem objects change
metadata 구조이기 때문에 OS가 자동으로 관리해야 안정성이 보장된다
 - Explicit writing is unnecessary and would be error-prone

File API (Attempt 2)

- `pread(const char path, void buf, off_t offset, size_t nbyte)`
 - Reads nbyte bytes starting from offset in the file specified by path.
- `pwrite(const char path, const void buf, off_t offset, size_t nbyte)`
 - Writes nbyte bytes starting from offset in the file specified by path.

Disadvantages

- Expensive directory traversal
매번 경로 탐색(directory traversal)을 반복해야 해!
 - Each pread / pwrite requires resolving the path → inode mapping.
- Goal
 - Minimize traversal cost — ideally perform path resolution only once

File Descriptor (fd)

Key Idea

- Perform the expensive path traversal only once during open()
open() 시: 경로 탐색 1회만 수행
- Store the resolved inode number inside a descriptor object (kept in memory)
Descriptor에 저장: inode 번호를 메모리에 보관
- Subsequent read() and write() calls operate via the descriptor, which maintains the current file offset
read()/write() 시: inode 번호로 직접 접근
read(fd, ...) → inode 3의 데이터만 읽기 (1 I/O)

Descriptor Object

- Contains:
 - inode number // 3 (bashrc의 inode)
 - current offset
 - access mode (read/write) // O_RDONLY, O_WRONLY, O_RDWR
 - reference count (for shared descriptors) // 몇 개의 프로세스가 공유 중?

Per-Process File Descriptor Table (Process FD Table)

- Each process maintains a file-descriptor table
각 프로세스마다 **FD 테이블** 보유
- The table stores pointers to open file-descriptor objects.

- File descriptors used in system calls are **simply integer indexes** into this table

FD는 단순한 정수!

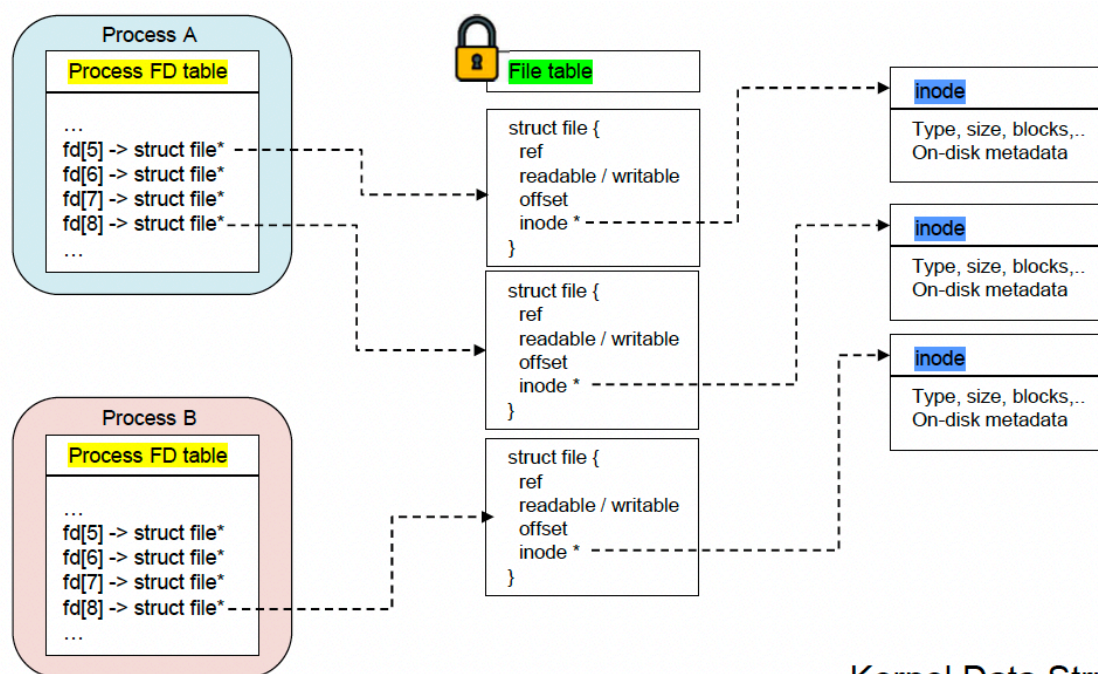
+) `read(fd, buf, 100);`

- 내부적으로
 - FD 테이블[3]에서 Descriptor 객체 찾기
 - 객체에서 inode 번호 확인 → 3
 - inode 3의 데이터 읽기

Well-Known File Descriptors

- stdin: 0
- stdout: 1
- stderr: 2

Kernel Data Structures (xv6 kernel)



Kernel Data Struct

Process FD table	File Table	inode
각 프로세스마다 독립적	시스템에 1개	시스템에 1개
메모리	메모리	디스크

fork()해도 FD Table: 2개 (부모 1개, 자식 1개) - 독립적 dup()하면 FD Table: 1개 (같은 프로세스니까) FD 번호: 다름 (3, 4) - struct file: 1개 - 공유	open()마다 새 entry dup()이나 fork()하면 공유	
---	---	--

xv6 struct file Definition

```
struct file {
    int ref;           // reference count
    char readable;     // opened for read?
    char writable;     // opened for write?
    struct inode *ip;  // underlying inode
    uint off;          // file offset
};
```

- Each process contains its own file descriptor table (proc→ofile[])
 - 각 프로세스는 자신의 FD 테이블을 가짐 - 각 프로세스는 `proc→ofile[]` 배열을 가짐
 - Entries in this table store pointers to struct file objects located in the global file table
 - 이 배열의 각 엔트리는 **struct file 객체의 포인터**를 저장

예시

Process A:

```
proc→ofile[0] → stdin의 struct file*
proc→ofile[1] → stdout의 struct file*
proc→ofile[2] → stderr의 struct file*
proc→ofile[3] → "file.txt"의 struct file*
```

- Thus:
 - A process does not hold file state directly
프로세스는 파일 상태를 직접 보관하지 않음
 - Multiple file descriptors (either within one process or across different processes) may refer to the same struct file entry, sharing offset and state 여러 파일 디스크립터가 같은 struct file을 참조 가능
 - Reference counting (ref) ensures **correct sharing semantics**
참조 카운트(ref)가 올바른 공유 의미를 보장

Global Open File Table (전역 파일 테이블)

```
struct {  
    struct spinlock lock;    // protects the table  
    struct file file[NFILE]; // all open files in the  
    system  
} ftable;
```

- Open files are stored in a fixed-size array, simplifying the design
 - 고정 배열이라 구현이 단순함
 - 메모리 할당/해제 불필요
- A single spinlock protects the entire table, which limits concurrency but keeps the kernel implementation small and easy to understand 하나의 spinlock으로 전체 테이블 보호

Sharing struct file Across Multiple Processes

- After fork(), the child inherits copies of the parent's FD table. But the FD entries still point to the same struct file objects.
- So, parent reads → offset moves; child reads next → continues from the new offset. They affect each other
- 서로 offset에서 영향 미침

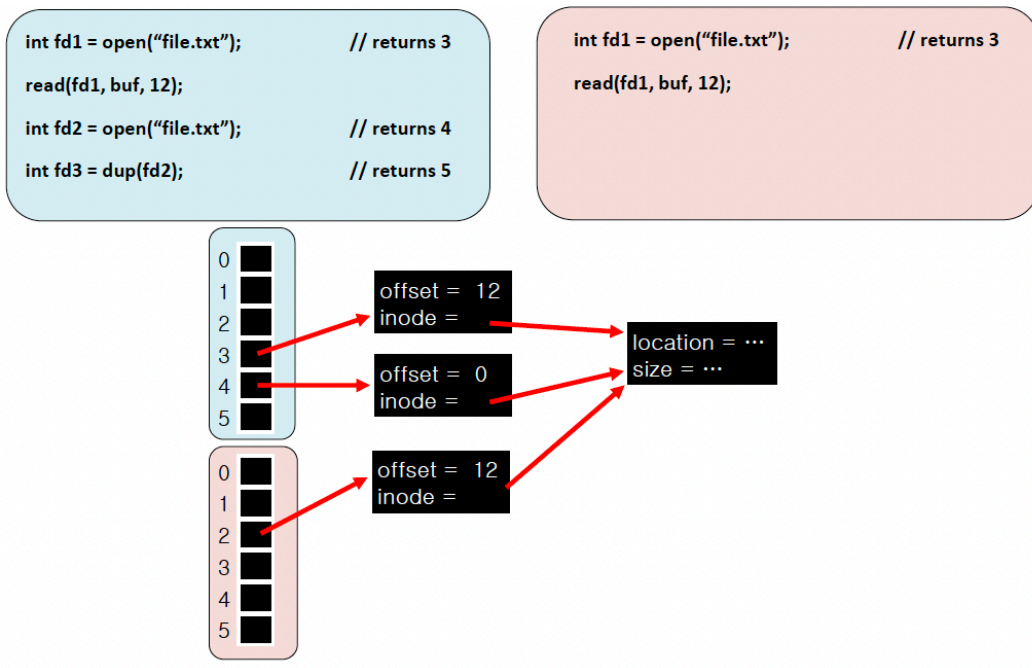
When using dup(), dup2(), or fcntl(F_DUPFD)

- These system calls create new FD numbers, but the underlying kernel struct file is shared.
- Thus, the offset is shared as well.

When is the offset NOT shared?

- Every open() call creates a new struct file
 - fd1 = open("x.txt", O_RDONLY); // struct file A (offset independent)
 - fd2 = open("x.txt", O_RDONLY); // struct file B (offset independent)
- Changing offset for fd1 does not affect fd2.

Code Snippet



p32 빨간색 블록: 2를 3으로 바꾸셈 3을 2로 바꾸던가

File API (Attempt 3)

- `int fd = open(char *path, int flag, mode_t mode)`
- `read(int fd, void *buf, size_t nbyte)`
- `write(int fd, void *buf, size_t nbyte)`
- `close(int fd)`
- Advantages:
 - String names
 - Hierarchical
 - Traverse once
 - Different offsets precisely defined

Deleting Files

- There is no direct system call whose purpose is "delete this file."
"파일을 삭제하라"는 직접적인 시스템 콜은 없다!
- An inode (and its associated file data) is garbage-collected when no references remain—neither from directory paths nor from open file

descriptors. inode와 파일 데이터는 **참조가 하나도 남지 않았을 때** 자동으로 회수됨.

- 참조의 두 가지 방식
 1. 디렉토리 경로에서 참조
 2. 열린 file descriptor에서 참조

참조 제거 방법

1. unlink() → 디렉토리 엔트리 제거, 그러나 inode를 즉시 해제하지는 X
2. close() 또는 프로세스 종료 → 파일 디스크립터 제거

Many File Systems

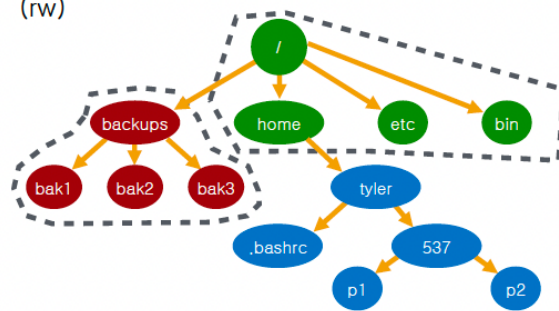
- Users often need to work with multiple file systems on a single machine
사용자는 하나의 컴퓨터에서 여러 파일 시스템을 사용해야 함
- Examples include:
 - main disk - 운영체제와 프로그램들
 - backup disk - 중요데이터 백업용
 - AFS (Andrew File System) - 네트워크 파일 시스템
 - USB thumb drives - 이동식 저장장치
- Question:
 - What is the most elegant and unified way for an operating system to support multiple file systems?
각 파일 시스템마다 다른 방식으로 접근해야 한다면? 운영체제가 여러 파일 시스템을 지원하는 가장 우아하고 통합된 방법은?
- Idea: Combine all file systems into a single unified namespace → a "super" file system
모든 파일 시스템을 하나의 통합된 네임스페이스로 결합 → 슈퍼 파일 시스템
- Example (output of mount):

- /dev/sda1 on /
ext4 (rw)
- /dev/sdb1 on /backups
ext4 (rw)
- AFS on /home
(w)

type (rw)

type

type afs



Communicating Requirements: fsync - 쓰기 버퍼링

- A file system keeps newly written data in memory temporarily
파일 시스템은 새로 쓴 데이터를 메모리에 임시 저장함
- Write buffering improves performance (why?)
- But what happens if the system crashes before these buffers are flushed?
버퍼가 플러시되기 전에 크래시되면
- If the application requires durability:
 - fsync(int fd)
 - forces all buffered data for that file to be written to disk
해당 파일의 모든 버퍼 데이터를 강제로 디스크에 쓰기
 - And (usually) instructs the disk to flush its own write cache as well
디스크의 자체 쓰기 캐시도 플러시하도록 지시 (보통)
 - This operation makes the data durable → 데이터를 영속적(durable)으로 만듦
- user buffer(memory) → kernel buffer(memory) → hdd?
- synchronous & asynchronous? → kernel demon이 asynchronous하게 30초마다?
 - synchronous
 - write(fd, data, 1000); // write()가 리턴될 때 = 디스크에 쓰여짐(느림)

- 디스크 쓰기 완료까지 기다림
- 안전하지만 느림
- asynchronous - 기본모드
 - write(fd, data, 1000); // write()가 리턴될 때 = 커널 버퍼에만 복사됨 (빠름!)
 - 커널 버퍼에 복사만 하고 즉시 리턴
 - 빠르지만 크래시 시 손실 가능

write() → User Buffer

↓ (복사)

Kernel Buffer (Page Cache)

↓ (자동 또는 fsync)

Disk Cache

↓ (fsync 시 플러시)

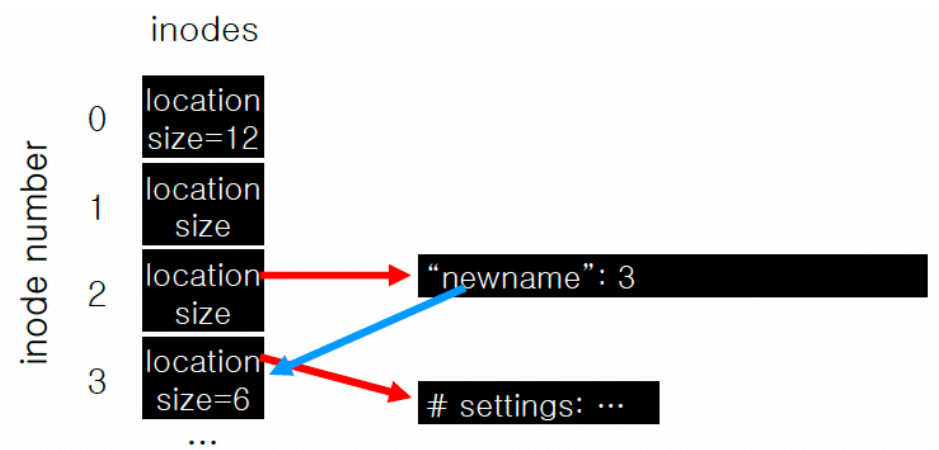
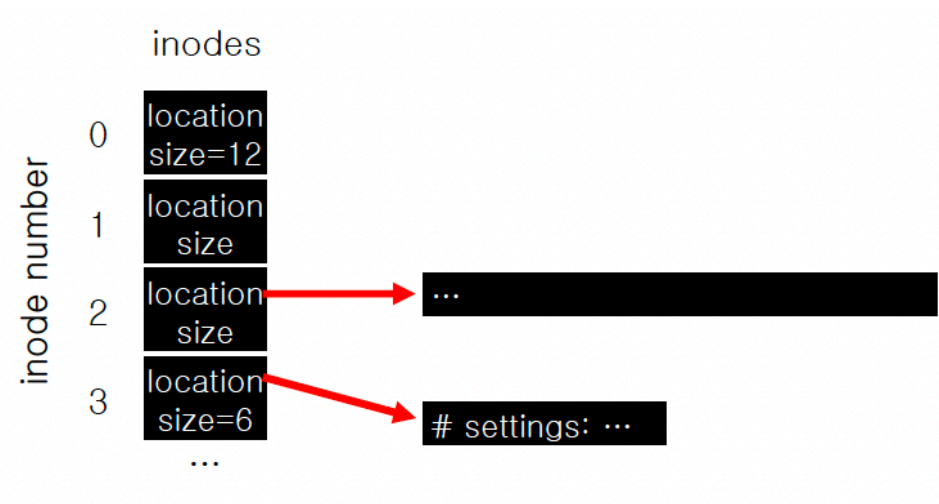
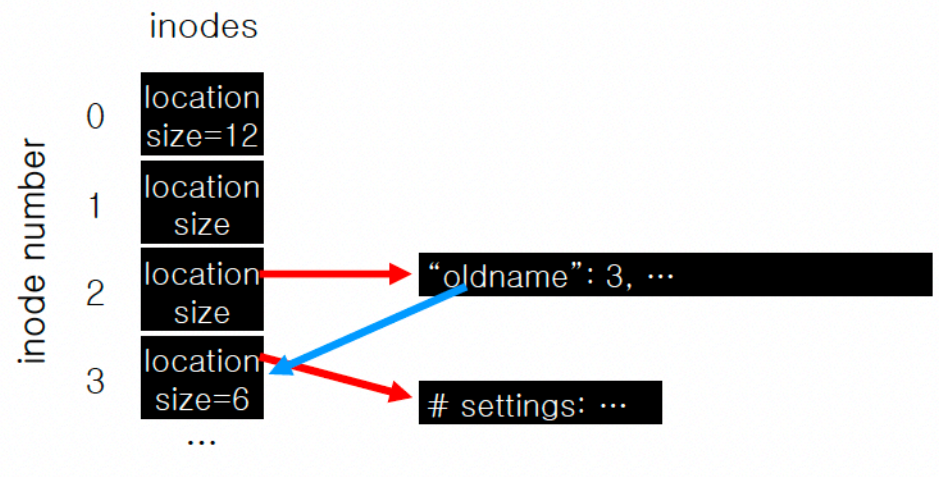
Disk Platter (영구!)

핵심: write()는 커널 버퍼까지만, fsync()가 디스크까지!

rename

- rename(char *old, char *new):
 - deletes an old directory entry to a file (old link)
기존 디렉토리 엔트리 삭제 (old link)
 - creates a new directory entry (new link) to the same file
새 디렉토리 엔트리 생성 (new link) - 같은 inode를 가리킴
- Only changes the file's name; it does not move or rewrite file data.
중요한 점: 파일 이름만 건드리지 파일 데이터는 안 건드립니다!
- What can go wrong if the system crashes at the wrong moment?
 - The file system must perform additional work to guarantee atomicity.

Example



Atomic File Update

- Suppose an application wants to update file.txt atomically.
- If the system crashes, it should either see only the old contents or only the new contents — never a mixture.

- Steps for an atomic update:
 - Write new data to file.txt.tmp
임시 파일에 새 데이터 쓰기
 - fsync file.txt.tmp to ensure data is fully on disk
fsync로 디스크에 확실히 쓰기
 - rename file.txt.tmp to file.txt, replacing the old version
rename으로 원자적 교체
- Why this works
 - rename() is atomic with respect to crashes.
 - If the system crashes during the rename operation, the file will appear as either the old name or the new name—no partial or in-between state can occur.

정리하면

질문: "보고서.txt를 최종보고서.txt로 이름 바꾸기"

답변: 그냥 `rename("보고서.txt", "최종보고서.txt")` 끝!

질문: "보고서.txt의 내용을 안전하게 업데이트"

답변:

1. "보고서.txt.tmp"에 새 내용 쓰기
2. fsync
3. rename("보고서.txt.tmp", "보고서.txt")
→ 이때 기존 "보고서.txt" 자동 삭제됨!

Summary

- Using multiple types of names provides both:
 - convenience
 - efficiency
- Mount and link mechanisms give the file system flexibility.

- Special system calls (e.g., fsync, rename) allow applications to communicate specific requirements - such as durability and atomicity - to the file system.